实验 6　工具调用、Agent 与 Harness 实现

—— AI 需求歧义审查器 • 审查档案分析助手

班　　级：软工 2 班

学　　号：24111302102

姓　　名：许文涛

实验日期：2026 年 11 月 8 日

仓库地址：https://github.com/Ventus-cn/AI_Request_Checker

Commit：7fe4988

学习通提交说明

- 提交平台：只向学习通对应的作业上传一份 PDF 文件。
- 文件命名：软工 2 班-24111302102-许文涛-实验 6-工具调用、Agent 与 Harness 实现.pdf。
- PDF 内容：任务说明、工具接口、Agent、Harness、执行日志和实验小结，附仓库链接与 Commit。
- 截止时间：以学习通作业设置为准。

1 实验目的

本实验要求在个人软件中完成工具调用、单 Agent 和运行时 Harness 的实现，并用测试记录证明控制流程真实生效。结合 AI 需求歧义审查器的已有能力，本实验的具体目的如下。

- 掌握工具接口定义、参数校验、权限边界与结果反馈：工具名由程序白名单约束，参数按 JSON Schema 校验，模型不能直接执行任意函数或命令。
- 实现由模型决定下一步、由程序执行工具的单 Agent 循环：工具的调用请求和结果按接口要求写回上下文，运行状态可查看、可终止。
- 编写运行时 Harness：负责上下文组织、工具分发、结果检查、人工确认、停止控制和运行日志，关键限制全部由程序执行。
- 用 5 类以上测试验证正常流程、参数错误、工具异常、步数与时间限制、人工确认，并形成可复核的实验报告。

2 实验内容

2.1 任务选择与边界

AI 需求歧义审查器已完成需求审查、追问版本、历史档案和 PDF 导出，审查档案保存在本机 H2 数据库中。本实验在现有产品上增加“审查档案分析助手”：用户用自然语言询问档案的风险情况，Agent 通过只读工具查询真实数据并给出数字可核对的结论；删除档案作为唯一的写操作，用来验证人工确认。

这样选择的原因是任务范围小、结果可判定：统计结论必须与数据库聚合结果逐项相等，不需要人工判断“写得好不好”；同时它直接服务于个人软件，历史档案和知识库都是现成数据，不必另造样例。

表 2-1　任务卡

项目	内容
任务目标	用自然语言查询审查档案的风险分布、单条档案详情和知识库规则，并给出可核对的结论
典型提问	① 现在有多少份档案，风险分布如何？② 最近一条档案最严重的问题是什么？③ 知识库里文件上传有什么限制？④ 删除某条测试档案
输入	自然语言问题（≤500 字符）+ 会话内工具结果
输出	中文回答：先结论数字，再给依据（工具统计或知识库片段编号），最后给建议
可用数据	H2 档案库（实验前 28 条）、rag/knowledge.json（10 个片段）
停止条件	① 回答通过程序检查；② 达到最大步数；③ 超过总时限；④ 等待确认被拒绝或超时；⑤ 发生不可恢复错误
成功标准	回答中的总数与高/中/低风险数量与 compute_risk_stats 逐项相等；引用知识库必须带片段编号；删除未经确认不执行

表 2-2　输入与预期结果

编号	提问	预期工具调用	预期结果
Q1	现在有多少份档案，风险分布如何？	compute_risk_stats(mode=all)	给出总数与三档计数，数字与工具一致
Q2	最近一条档案讲了什么，最严重的问题是什么？	list_reviews(limit=3) → get_review_detail(id)	指出档案 ID 与问题，不编造内容
Q3	知识库里文件上传有什么限制？	search_knowledge(q=…, k=3)	有结果时给出片段编号；无结果时说明资料不足
Q4	删除某条测试档案	delete_review(id) → 暂停等待确认	未确认不执行；确认后删除并回报
Q5	帮我查一下服务器磁盘占用	无工具可满足	明确说明无法执行，并建议可用问题

表 2-3　自动执行、必须确认与明确禁止

类别	内容
自动执行	list_reviews、compute_risk_stats、get_review_detail、search_knowledge——只读、幂等、无副作用
必须人工确认	delete_review：执行前展示档案 ID、创建时间、摘要和“不可恢复”提示；拒绝或 5 分钟未响应则不执行
明确禁止	执行模型生成的任意函数名、SQL 或 shell 命令；访问项目目录外文件；把 API Key 写入上下文或日志；批量删除；绕过确认直接写库；对写操作重复调用“碰运气”

表 2-4　程序预设与模型决定的边界

环节	由谁决定	实现位置
调用哪个工具、传什么参数	模型（输出 tool_calls）	QwenAgentModelClient
工具是否存在、参数是否合法	程序（白名单 + Schema 校验）	ToolRegistry、ArgumentValidator
工具执行与超时	程序（Future 限时）	AgentHarness.executeTool
是否继续循环、何时收尾	模型（无 tool_calls 即给出回答）	AgentHarness.loop
步数、总时限、重答次数	程序（到达即终止，模型无权延长）	AgentHarness、AnswerChecker
删除是否需要确认	程序（写工具强制挂起）	AgentRunStore
回答是否达标	程序（数字与引用核对）	AnswerChecker

2.2　工具接口与参数约束

系统共实现 5 个工具，其中 4 个只读、1 个写操作。所有工具都由 Spring 注入后注册进 ToolRegistry，形成唯一白名单；模型给出的函数名不在白名单内会直接返回 ARG_INVALID，程序不会反射调用任意方法，也不执行模型生成的命令。

表 2-5　工具清单

工具名	用途	输入	返回	超时	修改数据
list_reviews	列出档案摘要	limit 1－20；riskLevel 枚举	id、时间、摘要、风险、计数	5s	否
compute_risk_stats	风险统计	mode 枚举（all/三种审查视角）	total、high、medium、low、byType	5s	否
get_review_detail	单条档案详情	id（UUID）	档案字段与问题明细（截断）	5s	否
search_knowledge	检索知识库	q 2－100 字符；k 1－5	resultCount、chunks（含编号）	5s	否
delete_review	删除档案	id（UUID）	deleted、id	5s	是（需确认）

参数定义采用 OpenAI function 的 JSON Schema 结构，程序按 Schema 校验类型、范围、枚举、格式和多余字段。下面以 list_reviews 为例，完整定义见附录 A。

```
{
  "type": "object",
  "properties": {
    "limit":     {"type": "integer", "minimum": 1, "maximum": 20, "default": 5},
    "riskLevel": {"type": "string", "enum": ["all","high","medium","low"], "default": "all"}
  },
  "required": [],
  "additionalProperties": false
```

```
}
```

表 2-6　参数校验规则

检查项	规则	失败反馈（回传模型）
工具名	必须命中 ToolRegistry 白名单	ARG_INVALID：未知工具，并列出可用工具
类型	integer 必须是整数，string 必须是字符串	ARG_INVALID：需要整数/字符串
范围	limit 1－20、k 1－5、q 2－100 字符	ARG_INVALID：不能大于 20，收到 999
枚举	riskLevel、mode 必须命中枚举	ARG_INVALID：只能取 all/high/…
格式	id 必须匹配 36 位 UUID	ARG_INVALID：id 格式不正确
多余字段	additionalProperties=false	ARG_INVALID：不支持的参数 xxx
执行超时	单次工具 5 秒	TIMEOUT：已停止等待，状态可能未知

只读工具可重复调用，不产生副作用；delete_review 属于写操作，确认后只执行一次，若执行后出现异常则标记 STATE_UNKNOWN 并要求人工核对，不自动重试。工具超时只表示程序停止等待，不代表外部操作已经停止，这一点在 2.5 节单独说明。

接入 Agent 之前先做了直接测试：正确参数 list_reviews({"limit":3}) 返回 3 条真实档案；错误参数 list_reviews({"limit":999}) 被 ArgumentValidator 拒绝，回传“list_reviews.limit 不能大于 20，收到：999”，查询没有执行。两组结果分别由 ToolRegistryTest 和 AgentHarnessTest 固化，后文 2.6 节给出记录。

2.3　Agent 循环与状态

Agent 采用单轮循环：模型负责提出工具调用，程序负责校验、执行、回传和终止。循环图如下。

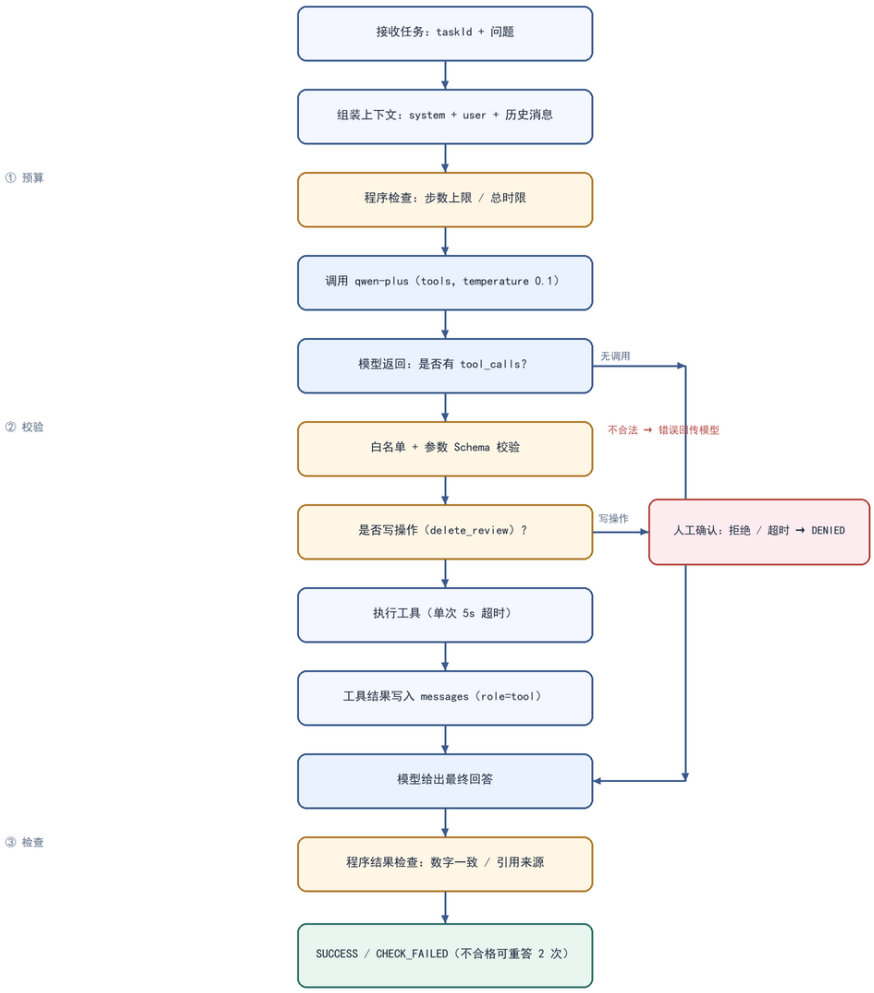


图 2-1　Agent 循环与状态（含参数校验、人工确认和停止检查位置）

调用与结果必须进入后续上下文，而不是只在控制台打印。assistant 的 tool_calls 原样回填，每个工具结果按 tool_call_id 对应一条 role=tool 消息，模型据此继续或收尾。

```
[system]    角色、任务边界、可用工具与禁止事项
[user]      现在有多少份档案，风险分布如何？
[assistant] tool_calls=[{id:"call_1", function:{name:"compute_risk_stats",
                                                arguments:"{\"mode\":\"all\"}"}}]
[tool]      {"tool_call_id":"call_1","name":"compute_risk_stats",
             "content":"{\"total\":28,\"high\":21,\"medium\":6,\"low\":1,...}"}
[assistant] 目前共有 28 份档案，其中高风险 21 份、中风险 6 份、低风险 1 份……
```

一步定义为模型的一次响应：无论这次响应发起 1 个还是多个工具调用，都只计一步；工具执行不单独计步，但计入总时限；单步最多允许 3 个工具调用，超出部分直接拒绝。这样计数可以避免模型用并行调用绕过步数限制。

表 2-7　程序级运行限制

参数	取值	到达后的行为
最大步数	6 步	终止并说明“已达最大步数，结论不完整”
模型超时	30 秒/次	本次调用失败，不再重试，返回 FAILED
工具超时	5 秒/次	向模型回传 TIMEOUT，允许换工具或说明原因
总时限	60 秒/次运行	下一步开始前检查，超时立即终止
重答次数	2 次	仍不合格则 CHECK_FAILED
单步工具数	3 个	超出部分拒绝并回报
确认有效期	5 分钟	超时不执行，任务失效

结果检查同样由程序执行，不采信模型“已完成”的自述，规则有三条：第一，如果本次运行调用过 compute_risk_stats，最终回答必须包含总数和高/中/低风险四个数字，且与最近一次工具结果相等；第二，调用过 search_knowledge 且有结果时，回答必须出现 KB-xxx 编号；第三，没有检索却引用 KB 编号视为无依据引用，要求模型删除或重新检索。检查不通过时把原因作为系统消息发回，最多重答 2 次。

2.4 运行时 Harness 设计与实现

Harness 是本实验的外围程序，负责组织模型和工具之间的每一次交互。模块划分如下图，右侧是工具与模型通道，中间是编排核心，下方是日志、确认暂存和结果检查。

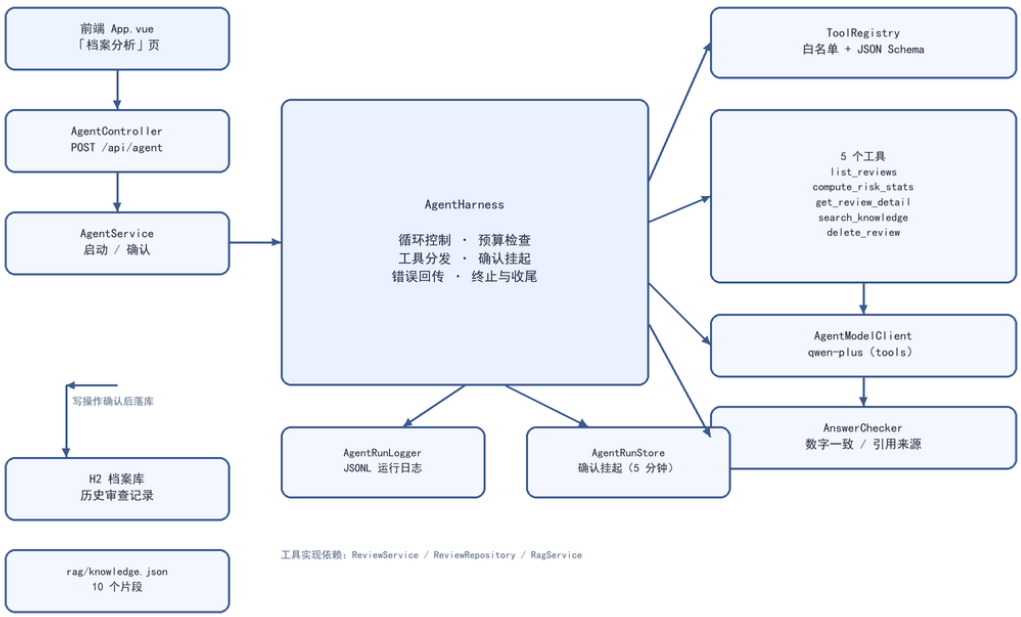


图 2-2 运行时 Harness 模块与依赖

表 2-8 Harness 模块职责

类	职责
AgentController	REST 入口，校验问题长度与确认参数，统一返回 400/404
AgentService	任务编排入口：启动运行、恢复确认
AgentHarness	循环控制、预算检查、工具分发、确认挂起与恢复、错误回传、终止
ToolRegistry	白名单：未命中的工具名直接拒绝
ArgumentValidator	按 JSON Schema 校验类型、范围、枚举、格式和多余字段
AgentModelClient	模型通道抽象；QwenAgentModelClient 调用 qwen-plus
AnswerChecker	数字一致性、知识库引用与无依据引用检查
AgentRunLogger	逐事件写 JSONL；参数与结果截断，密钥不落盘
AgentRunStore	保存等待确认的运行，5 分钟 TTL

上下文组织遵循“数据必须来自工具”的原则：系统提示只写角色、边界和输出要求，不放任何示例数字；历史工具结果按 tool_call_id 排列；单条工具结果进入上下文前截断到 2000 字符，避免上下文无限增长。确认流程被建模成显式状态：写操作不直接执行，Harness 先返回 NEED_CONFIRM 和影响说明，用户同意后才从断点继续，拒绝则直接以 DENIED 结束并明确“未删除任何档案”。

2.5 失败处理与可观测性

每次运行生成一个任务编号，逐事件写入 backend/logs/agent-runs/<taskId>.jsonl。日志只记录工具名、参数摘要、结果摘要、错误类型、步数、耗时和最终状态，不记录 API Key、完整需求文本或完整外部响应。

表 2-9 运行日志字段

字段	说明
taskId	yyyyMMdd-HHmmss-6 位随机，与前端展示一致
ts / step / phase	时间；步号；context/model/tool/confirm/check/finish
tool / args / result	工具名；参数摘要（≤120 字）；结果摘要（≤200 字）
status / errorType	ok 或 error；ARG_INVALID、NOT_FOUND、TOOL_EXCEPTION、TIMEOUT、STATE_UNKNOWN 等
durationMs / tokens	本事件耗时；模型返回的 prompt/completion token
finalStatus	SUCCESS、DENIED、STEP_LIMIT、TIMEOUT、CHECK_FAILED、FAILED

一次真实运行的日志（节选自任务 20261108-162558-c8c0a9，已省略 token 细节）：

```
{"taskId":"20261108-162558-c8c0a9","step":0,"phase":"context","status":"ok",
 "tools":"compute_risk_stats,delete_review,get_review_detail,list_reviews,search_knowledge"}
{"taskId":"20261108-162558-c8c0a9","step":1,"phase":"model","status":"ok","durationMs":900}
{"taskId":"20261108-162558-c8c0a9","step":1,"phase":"tool","tool":"compute_risk_stats",
```

```
 "status":"ok","durationMs":27,"result":"{\"total\":28,\"high\":21,...}"}
{"taskId":"20261108-162558-c8c0a9","step":2,"phase":"model","status":"ok","durationMs":2486}
{"taskId":"20261108-162558-c8c0a9","step":2,"phase":"check","status":"ok","result":"ok"}
{"taskId":"20261108-162558-c8c0a9","step":2,"phase":"finish","status":"ok","finalStatus":"SUCCESS"}
```


表 2-10　错误分类与用户可见反馈

情形	内部状态	页面/回答
参数错误	ARG_INVALID 回传模型	模型换参数重试；仍失败则说明参数不合法并停止
工具异常/记录不存在	TOOL_EXCEPTION / NOT_FOUND	说明查不到该档案，不编造数据
工具或模型超时	TIMEOUT	已停止等待；后台可能仍在执行，状态未知，请核对
达到步数上限	STEP_LIMIT	结论不完整，并给出已获得的工具结果
用户拒绝确认	DENIED	已取消，未删除任何档案
写操作执行后异常	STATE_UNKNOWN	结果不确定，请到列表核对；不自动重试
结果校验失败	CHECK_FAILED	统计数字与工具结果不一致，已终止

2.6　测试与结果分析

后端共 33 个自动化测试全部通过，其中本实验新增 17 个：AgentHarnessTest 11 个（模型使用测试桩，覆盖失败与边界），ToolRegistryTest 6 个（白名单与参数 Schema）。原有 16 个测试不受影响。真实流程另外执行了 5 次，全部调用 qwen-plus 与实际工具。

表 2-11　五类测试覆盖与结果

编号	类别	输入/操作	实际结果
TA-01	正常完成（真实模型）	询问档案风险分布	SUCCESS；调用 compute_risk_stats；回答 30 份、高 21、中 6、低 3，与工具结果一致
TA-02	参数错误（测试桩）	模型传 limit=999	程序拒绝并回传 ARG_INVALID，查询未执行；模型改口径后结束
TA-03	工具异常（测试桩）	查询不存在的 UUID	返回 NOT_FOUND，回答说明查不到，不编造
TA-04	步数上限（测试桩）	max-steps=1，模型继续请求工具	STEP_LIMIT，回答说明结论不完整
TA-05	时间限制（测试桩）	慢工具 + 50ms 工具超时	TIMEOUT 回传模型，运行继续并收尾
TA-06	人工确认-同意（真实）	删除测试档案并确认	执行删除，档案从列表消失，日志 approved→tool ok
TA-07	人工确认-拒绝（真实）	删除测试档案并拒绝	DENIED，回答“未删除”，档案仍可查询
TA-08	结果校验（测试桩）	回答写错数字 / 无依据引用	触发重答；仍错则 CHECK_FAILED；引用被要求删除

TA-09	未知工具（测试桩）	调用 delete_all	ARG_INVALID：未知工具，拒绝执行

表 2-12　真实运行记录

任务编号	问题	工具调用	最终状态
20261108-162656-e75830	档案总数与风险分布	compute_risk_stats	SUCCESS
20261108-162700-2bdd63	删除测试档案 f1e24e91	delete_review（确认后）	SUCCESS
20261108-162705-94bcdc	删除测试档案 8b1d538f	delete_review（拒绝）	DENIED
20261108-162818-968b42	知识库文件上传限制	search_knowledge（0 条）	SUCCESS（说明资料不足）
20261108-162820-5fcb93	最近 3 条档案摘要	list_reviews(limit=3)	SUCCESS

说明：为验证删除流程，实验前临时创建了 2 条测试档案，因此统计运行显示 30 份；确认删除后剩 29 份，被拒绝删除的一条保留。

开发和测试过程中发现并修复了 8 个问题，可以说明“AI 初稿—人工修正—测试通过”的过程：

- 泛型捕获：对 Map<?,?> 调用 getOrDefault 编译报错，改为先取 Object 再判断空值。
- lambda 捕获：参数解析分支的局部变量被重新赋值，导致“必须是最终变量”错误，改用独立变量承接。
- 测试 Schema 缺省值：测试用例的 riskLevel 没有写 default，断言失败，补上默认值。
- 单例状态污染：前一个用例把 maxSteps/totalTimeoutMs 改小，后续用例误判超时，在每个用例前重置预算。
- 增量编译残留：target/test-classes 中的旧 class 引用未打包的类，出现 NoClassDefFoundError，执行 clean 后恢复。
- 无依据引用：真实模型中曾出现未检索却附带 [KB-003] 的情况，补充“无检索不得引用”的检查规则。
- 提示词引导：最初要求“先说明再调用”，模型改为反问用户要 ID，删除流程走不通；改为明确用户要求后直接调用、由程序确认。

• 未修复的局限：知识库按整段汉字分词，“文件上传限制”检索不到“文件上传”片段，出现 0 条结果。模型按规则回答“资料不足”，没有编造，但检索召回仍需改进。

从定位看，上述问题分别落在参数（缺省值）、工具（数据解析与检索）、检查规则（引用校验）、控制流程（单例预算、增量编译）和模型决策（提示词引导）五个层面，说明“让模型自己保证正确”并不可靠，程序侧的校验和状态管理才是可测试的部分。

3 操作记录

以下截图均在真实运行的前后端上截取：后端 Spring Boot 3.3.5（端口 8080），前端 Vite（端口 5173），模型为 qwen-plus。截图编号与正文引用一致。

表 3-1 截图清单

编号	内容	证明什么
图 3-1	首页（侧边栏新增“档案分析”）	功能已集成进现有产品
图 3-2	档案分析页初始状态	输入区、示例问题与运行按钮
图 3-3	正常分析结果与工具轨迹	真实调用模型与 compute_risk_stats，回答含数字
图 3-4	删除前的人工确认弹窗	写操作执行前展示影响并等待确认
图 3-5	确认后删除完成	确认→执行→回答的完整闭环
图 3-6	拒绝删除后的状态	拒绝不执行，状态为“已取消”
图 3-7	审查档案列表	档案已删除、另一条仍在，列表与数据库一致



图 3-1　首页：侧边栏新增“档案分析”入口

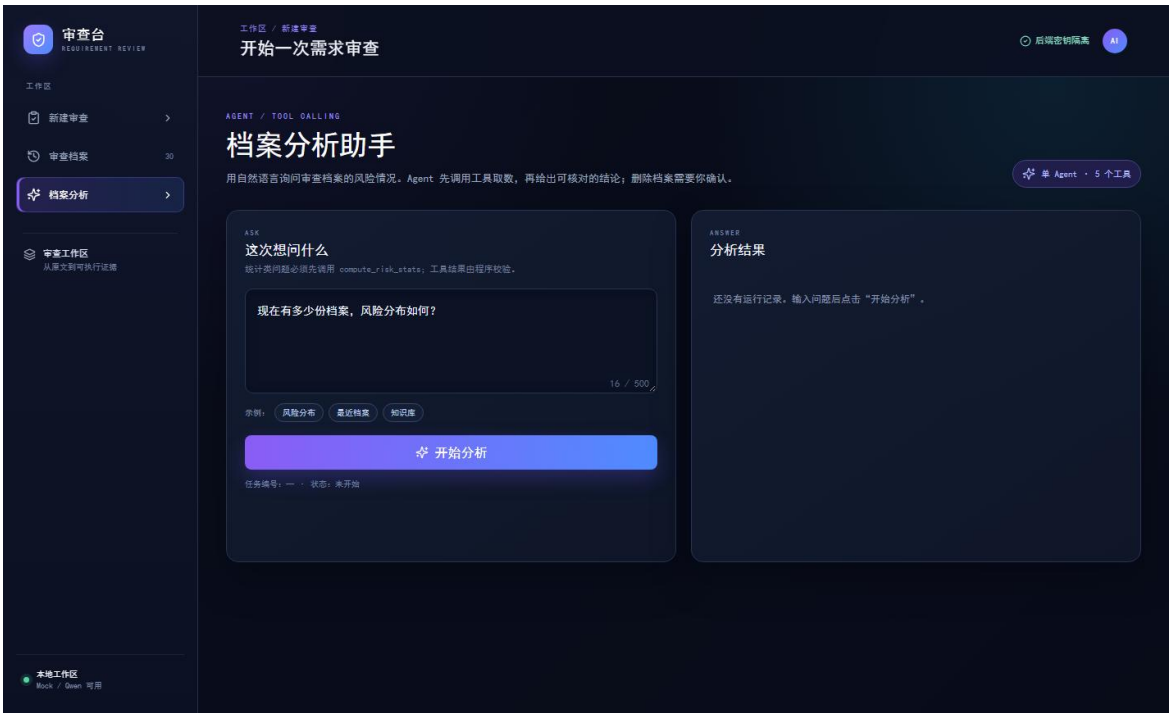


图 3-2　档案分析页：问题输入、示例问题与运行按钮

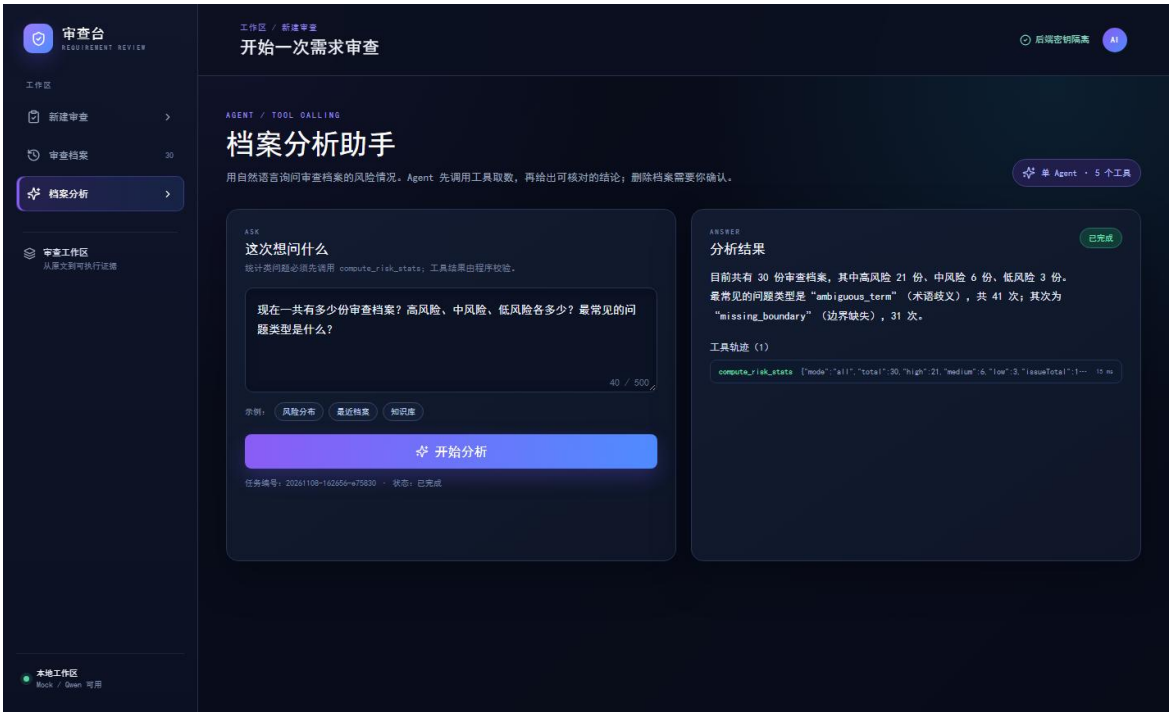


图 3-3　正常分析：回答给出 30 份档案与高/中/低风险数量，下方为工具轨迹

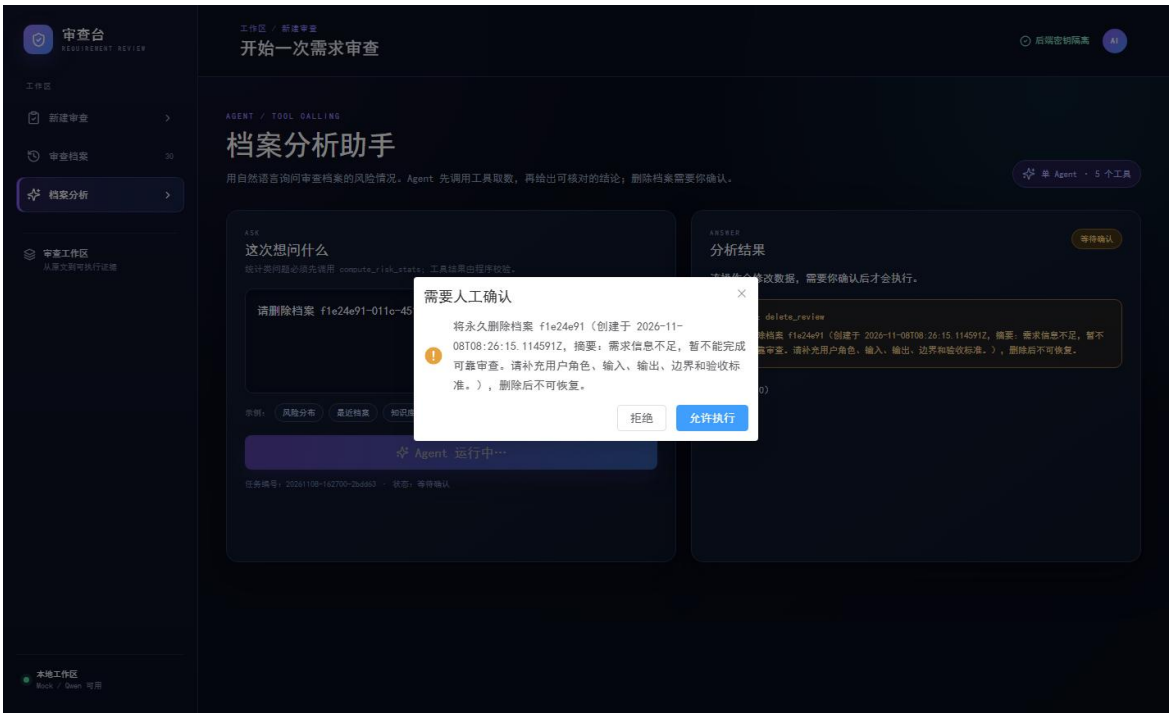


图 3-4　删除档案前弹出人工确认，展示档案摘要与“删除后不可恢复”

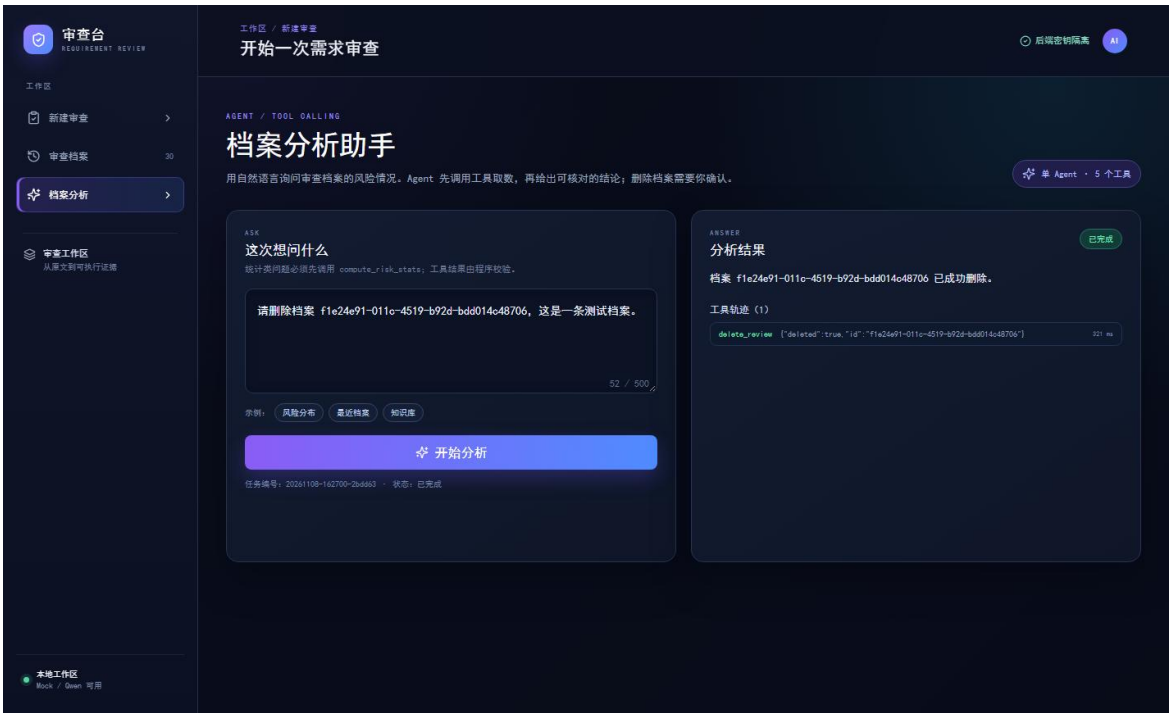


图 3-5　确认执行后：档案被删除，运行状态为“已完成”

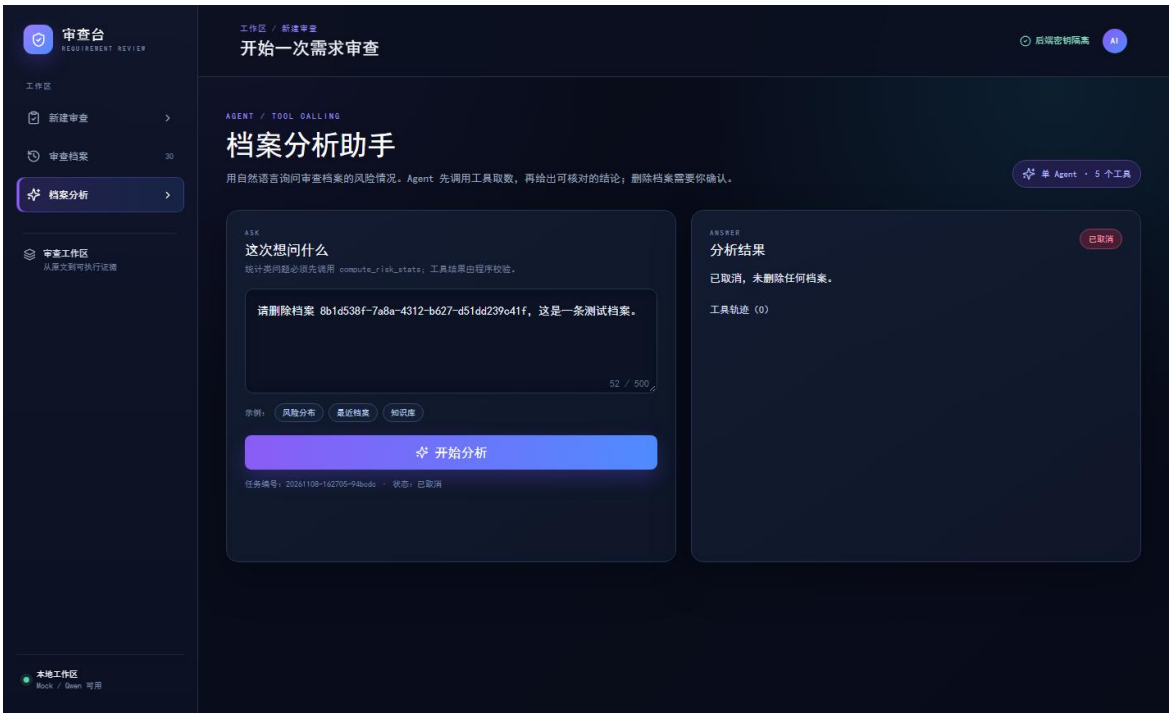


图 3-6　拒绝确认：状态为“已取消”，回答说明未删除



图 3-7　审查档案列表：被删除的档案不再出现，拒绝删除的档案仍在

代码与运行日志：工具、Agent、Harness 和测试代码位于 backend/src/main/java/com/example/ambiguity/agent 与 backend/src/test/java/com/example/ambiguity/agent；运行日志位于 backend/logs/agent-runs，每个任务一个 JSONL 文件。前端改动集中在 frontend/src/App.vue 的“档案分析”页与 style.css 的 .agent-* 样式。

表 3-2　仓库与版本证据

项目	内容
仓库地址	https://github.com/Ventus-cn/AI_Request_Checker
实验 6 Commit	7fe4988（完整哈希 7fe49885cc8fc927bafde32925ccdae817ecd0ec）
提交说明	实验 6：新增审查档案分析 Agent（工具调用、Harness、测试与运行日志）
推送结果	f5af077..7fe4988 main -> main
验证命令	cd backend && mvn test（33 个测试通过，BUILD SUCCESS）；cd frontend && npm run build（类型检查与构建通过）
运行说明	实验文档/TEST6/运行说明.md

4　实验小结

本次实验在 AI 需求歧义审查器上完成了“审查档案分析助手”：5 个白名单工具、单 Agent 循环、运行时 Harness、人工确认、结果检查和 JSONL 运行日志，并通过 33 个自动化测试和 5 次真实运行验证。Agent 已经作为产品的一个页面和接口使用，不是独立练习。

实现过程中体会最深的是三点。第一，能用程序做的事不要交给提示词：白名单、Schema、步数、超时和确认流程一旦写进代码，模型就无法绕过，测试也才有稳定的断言对象。第二，人工确认必须建模成显式状态，而不是让模型在回答里“请用户确认”，否则无法保证写操作真的没有执行。第三，日志和结果检查是排错的核心：真实运行中出现过无依据引用和检索 0 条的情况，正是日志里的 phase、错误类型和检查结果让问题可定位。

当前局限也很明确：知识库仍用整段汉字匹配，口语化提问召回不足；上下文只做了截断，没有摘要压缩；工具串行执行，尚未利用并行调用；前端仍是单文件页面结构。后续计划把检索改为二字词或向量匹配、为长上下文增加摘要、把“档案分析”拆成独立组件，并在需要多人使用时补充鉴权和审计落库。

附录 A　工具参数定义

以下为 5 个工具的完整参数定义（OpenAI function.parameters 结构）。

```
list_reviews
{"type":"object","properties":{
  "limit":{"type":"integer","minimum":1,"maximum":20,"default":5},
  "riskLevel":{"type":"string","enum":["all","high","medium","low"],"default":"all"}},
 "required":[],"additionalProperties":false}


compute_risk_stats
{"type":"object","properties":{
  "mode":{"type":"string","enum":["all","全面审查","可测试性","安全与权限"],"default":"all"}},
 "required":[],"additionalProperties":false}


get_review_detail / delete_review
{"type":"object","properties":{
  "id":{"type":"string","pattern":"^[0-9a-fA-F-]{36}$"}},
 "required":["id"],"additionalProperties":false}


search_knowledge
{"type":"object","properties":{
  "q":{"type":"string","minLength":2,"maxLength":100},
  "k":{"type":"integer","minimum":1,"maximum":5,"default":3}},
 "required":["q"],"additionalProperties":false}
```

附录 B　运行说明

环境：JDK 17、Maven 3.9+、Node.js 20+；在 backend/.env.local 配置 AI_MODE=qwen、DASHSCOPE_API_KEY 和 QWEN_MODEL=qwen-plus（该文件不入库）。

```
cd backend  && ./run-local.ps1        # 后端 8080
cd frontend && npm install && npm run dev   # 前端 5173
# 打开 http://localhost:5173 ，左侧“档案分析”


POST /api/agent/runs              {"question":"现在有多少份档案？"}
POST /api/agent/runs/{id}/confirm {"approved":true}
```

可调参数：AGENT_MAX_STEPS（默认 6）、AGENT_TOTAL_TIMEOUT_MS（60000）、AGENT_TOOL_TIMEOUT_MS（5000）、AGENT_MAX_ANSWER_RETRIES（2）、AGENT_CONFIRM_TTL_MS（300000）、AGENT_LOG_DIR（./logs/agent-runs）。

附录 C　关键代码节选

参数校验（ArgumentValidator）：

```
for (String key : input.keySet())
  if (!properties.containsKey(key))
    throw new ToolException(ARG_INVALID, "不支持的参数：" + key);
for (String key : required)
  if (input.get(key) == null)
    throw new ToolException(ARG_INVALID, "缺少必填参数：" + key);
if (definition.get("maximum") instanceof Number max && result > max.intValue())
  throw new ToolException(ARG_INVALID, name + " 不能大于 " + max.intValue());
```

循环与预算检查（AgentHarness）：

```
while (true) {
  if (run.step >= maxSteps)      return finish(run, "STEP_LIMIT", ...);
  if (run.expired())             return finish(run, "TIMEOUT", ...);
  run.step++;
  var result = model.complete(run.messages, registry.schemas());
  var calls  = toolCalls(result.message());
  if (calls.isEmpty()) { /* 结果检查 → SUCCESS / 重答 / CHECK_FAILED */ }
  for (var call : calls) handleCall(run, call);   // 校验、确认、执行、回传
}
```

结果检查（AnswerChecker）：

```
if (run.lastStats != null)
  for (String key : List.of("total","high","medium","low"))
    if (!containsNumber(answer, String.valueOf(run.lastStats.get(key))))
      return new Result(false, "回答缺少统计数字：" + key, "请补充并与工具结果一致");
if (run.lastKnowledgeCount == 0 && answer.matches("(?s).*KB-\\d+.*"))
  return new Result(false, "回答引用了未检索的知识库编号", "请删除该编号或先检索");
```